

Microtaller 03 — Actividad evaluativa

El presente trabajo da respuesta a la actividad evaluativa del Microtaller 03 sobre patrones, modelado del diseño, interfaz de usuario y diseño de componentes. Para fundamentar cada respuesta con un caso concreto se toma como referencia el proyecto **Pawcare: App móvil para atención veterinaria a domicilio**, cuyo backend está construido en Ruby on Rails (modo API) y cuya aplicación móvil se desarrolla en React Native con Expo SDK 56. De este modo, los conceptos teóricos se ilustran con decisiones de diseño reales del sistema.

1. Selección del patrón informático y especificación de sus elementos

De las tres imágenes propuestas, la que corresponde a un **patrón informático** es la **imagen 1**, que representa el patrón arquitectónico **Modelo–Vista–Controlador (MVC, Model–View–Controller)**. Las imágenes 2 y 3 no son patrones: la imagen 2 muestra un conjunto de monitores con gráficos bursátiles (un entorno de visualización de datos, no un esquema de organización del software) y la imagen 3 ilustra una topología de **computación en la nube** (un modelo de despliegue de infraestructura, no un patrón de diseño de software).

Un patrón informático es una **solución reutilizable y probada para un problema recurrente** dentro de un contexto determinado; describe cómo organizar responsabilidades entre las partes del software, no un producto ni una infraestructura concreta. La imagen 1 cumple esta definición porque separa la aplicación en tres responsabilidades bien delimitadas.

Los **elementos** del patrón MVC son:

- **Modelo (Model).** Encapsula los datos y la lógica de negocio; representa el estado del dominio y las reglas que lo gobiernan, con independencia de cómo se muestren. En Pawcare corresponde a los modelos de ActiveRecord (`Pet`, `Appointment`, `Consultation`, `Payment`) y a sus validaciones e invariantes.
- **Vista (View).** Es la representación de la información para el usuario; toma los datos del modelo y los presenta. En un backend API la "vista" son las respuestas **JSON** serializadas que consume el cliente; en la app móvil, las pantallas y componentes de React Native que renderizan esos datos.
- **Controlador (Controller).** Recibe las solicitudes del usuario, coordina al modelo y selecciona la vista de respuesta; actúa como intermediario. En Pawcare son los controladores delgados de Rails, que delegan la lógica en clases de acción que devuelven un resultado explícito (`Result.success(...) / `Result.failure(error:)`).

Razonamiento. El flujo de la imagen 1 (la vista emite una acción del usuario, el controlador la procesa y actualiza el modelo, y el cambio del modelo se refleja de vuelta en la vista) describe exactamente la separación de responsabilidades de MVC. Esta separación es la que hace al patrón valioso: permite cambiar la presentación sin tocar las reglas de negocio, probar el modelo de forma aislada y reutilizar la misma lógica para distintos clientes. En Pawcare, gracias a MVC el mismo backend sirve a los dominios Público, Dueño y Administrador a través de las 157 rutas del API sin duplicar la lógica de negocio.

2. Ejemplos de patrones de diseño según su concepto

Un **patrón de diseño** es una descripción de una solución general y reutilizable a un problema de diseño que se repite; no es código terminado, sino una plantilla de cómo

estructurar clases y objetos para resolverlo (Gamma et al., 1994). Los patrones clásicos del "Gang of Four" se clasifican en tres familias según el tipo de problema que resuelven: **creacionales** (cómo se crean los objetos), **estructurales** (cómo se componen) y **de comportamiento** (cómo colaboran y se comunican). A continuación se ejemplifica cada categoría y se vincula con Pawcare.

Categoría	Patrón	Concepto	Ejemplo en Pawcare
Creacional	Singleton	Garantiza una única instancia con acceso global.	Cliente HTTP centralizado de la app móvil: una sola instancia configura `baseURL`, token y reintentos.
Creacional	Factory Method	Delega la creación de objetos a un método especializado.	Construcción de "recordatorios" (cita, vacuna, desparasitación) a partir de un tipo.
Estructural	Adapter	Adapta una interfaz a la que el cliente espera.	Capa de servicios que adapta el JSON del API a los tipos de TypeScript del cliente.
Estructural	Facade	Ofrece una interfaz simple sobre un subsistema complejo.	Hooks reutilizables que ocultan la orquestación de servicios HTTP a las pantallas.
Comportamiento	Observer	Notifica a los interesados cuando cambia un estado.	Recordatorios y confirmaciones de citas que reaccionan a cambios de la agenda.
Comportamiento	Strategy	Encapsula algoritmos intercambiables.	Cálculo de costos por tipo de servicio (consulta, hospedaje, grooming, cirugía).

Además de los patrones GoF, el proyecto aplica **patrones arquitectónicos** de mayor escala: **MVC** y **API RESTful** en el backend, **arquitectura por capas** en el cliente móvil (navegación → pantallas → componentes → servicios → hooks → tipos) y el **patrón Acción/Resultado** (Action/Result), que mantiene los controladores delgados encapsulando la lógica en clases que devuelven `Result.success` o `Result.failure`. El criterio para elegir un patrón es siempre el mismo: existe un problema recurrente y el patrón aporta una solución probada que mejora la mantenibilidad y la claridad.

3. Relación del modelado del diseño con el ciclo de vida y el producto

El **modelado del diseño** es la fase del ciclo de vida del software que traduce el "qué" del análisis de requisitos en el "cómo" de la construcción; produce los modelos (de datos, de arquitectura, de componentes y de interfaz) que sirven de plano para la programación. Su relación con el ciclo de vida y con el producto es de **punto y trazabilidad**:

- **Con el ciclo de vida.** El diseño se ubica entre el análisis de requisitos y la implementación, y alimenta luego las fases de pruebas, despliegue y mantenimiento. En Pawcare el ciclo de vida combina un **diagnóstico participativo iterativo** (preparación, recopilación, análisis, validación y desarrollo) con el marco ágil **Scrum**: los requisitos priorizados se modelan, se construyen en sprints cortos y se validan con el consultorio, de modo que el modelo de diseño evoluciona de forma incremental con la retroalimentación de los informantes clave.
- **Con el producto.** El diseño determina los atributos de calidad del producto final. En Pawcare los modelos se alinean con la norma **ISO/IEC 25010** (usabilidad, rendimiento, seguridad, mantenibilidad, fiabilidad), de manera que las decisiones de diseño —arquitectura de tres capas, operación offline-first, seguridad por roles— se reflejan directamente en las características del producto entregado.

El vínculo se hace explícito mediante la **trazabilidad del diseño**: cada requisito funcional se encadena con las rutas del API que lo implementan, las pantallas que lo presentan y los componentes de UI que lo construyen (cadena *requisito → ruta → pantalla → componente*). En el proyecto, 156 de las 157 rutas móviles están mapeadas a un requisito. Así, un buen modelado del diseño asegura que ninguna

necesidad quede huérfana y que el producto construido corresponda fielmente a lo analizado, reduciendo el retrabajo a lo largo del ciclo de vida.

4. La interfaz de usuario y su importancia

La **interfaz de usuario (UI)** es el conjunto de elementos visuales, táctiles y de interacción a través de los cuales una persona se comunica con el sistema: pantallas, controles, mensajes, gestos y respuestas. Es la parte del software con la que el usuario tiene contacto directo y, por tanto, la que determina su percepción de calidad. Conviene distinguirla de la **experiencia de usuario (UX)**, que es más amplia e incluye lo que la persona siente y logra al usar el producto; la UI es el medio concreto que materializa esa experiencia.

Su **importancia** se puede resumir en varios puntos:

- **Usabilidad y adopción.** Una interfaz clara reduce la curva de aprendizaje y los errores. En Pawcare se fija como meta menos de dos horas de formación y completar las tareas comunes en 3 a 5 interacciones, para que tanto la Dra. Génesis como los dueños adopten la herramienta.
- **Accesibilidad.** El enfoque **mobile-first** y las guías de **Material Design** establecen áreas táctiles de al menos 48 dp, tipografía de 16 px o más y respeto de las áreas seguras, de modo que la app sea usable a una mano y por personas de distintos perfiles.
- **Consistencia y confianza.** El uso de un **sistema de tokens de diseño** (colores, espaciado, tipografía, sombras) con modo claro y oscuro garantiza una apariencia coherente en todas las pantallas, lo que transmite profesionalismo y reduce la carga cognitiva.

- **Prevención y recuperación de errores.** Estados de carga, vacío y error bien diseñados evitan que el usuario quede sin orientación, algo crítico en un servicio que ocurre fuera de la clínica y con conectividad variable (operación offline).

En síntesis, la interfaz de usuario es decisiva porque por buena que sea la lógica interna, el valor del sistema solo llega al usuario a través de una interfaz comprensible, accesible y consistente.

5. Diseño de componentes partiendo de la definición

Un **componente** es una unidad de software cohesiva, con responsabilidad única, interfaz bien definida y reutilizable, que puede combinarse con otros para construir el sistema. **Diseñar un componente partiendo de la definición** significa especificar primero su **contrato** —qué recibe (props), qué estados maneja y qué tokens de diseño usa— y solo después implementarlo. Este enfoque favorece la reutilización y la mantenibilidad (criterio RNF-MAINT-001 de Pawcare).

A partir del **design system** de Pawcare se diseñó un conjunto de componentes de UI reutilizables. Como entregable práctico de este punto se implementaron en **React Native (Expo SDK 56, TypeScript)** —la tecnología real de la app móvil— con los tokens del proyecto, usando primitivos nativos (``View``, ``Text``, ``Pressable``, ``TextInput``, ``StyleSheet``). El código completo se incluye en el **Anexo A** y en el archivo ``docs/microtaller-03-componentes.tsx``. Los componentes diseñados son:

- **Button** — acción del usuario. Variantes ``primary``, ``secondary``, ``outline``, ``destructive``, ``ghost`` y tamaños ``sm``/``md``/``lg``, con área táctil mínima de 44 px (``Pressable``).

- **TextField** — entrada de datos con `label`, `placeholder`, estado de error y tipografía de 16 px (evita el zoom automático en iOS).
- **Badge** — etiqueta de estado del dominio (`success`, `warning`, `destructive`, `primary`).
- **StatCard** — métrica numérica del dashboard (valor + etiqueta).
- **AppointmentCard** — fila compuesta que combina avatar, texto y *badge* de estado, usada en las citas del dueño y en la agenda del administrador.

La **definición** del componente Button, por ejemplo, fija su contrato antes de implementarlo: variante visual, tamaño, ancho completo opcional y estado presionado.

Su implementación en React Native, usando los tokens compartidos, es la siguiente:

```
// Definición: variante (primary|secondary|outline|destructive|ghost),
// tamaño (sm|md|lg), fullWidth opcional, área táctil ≥ 44 px.
export function Button({ label, variant = 'primary', size = 'lg', ...rest }:
ButtonProps) {
  const v = buttonVariantStyles[variant];
  return (
    <Pressable
      accessibilityRole="button"
      style={({ pressed }) => [
        styles.btn,
        { minHeight: SIZE_HEIGHT[size], backgroundColor: v.bg, borderColor:
v.border },
        pressed && { opacity: 0.9 },
      ]}
      {...rest}
    >
      <Text style={[styles.btnLabel, { color: v.fg }]}>{label}</Text>
    </Pressable>
  );
}
```

Diseñar así —de la definición a la implementación— permite que cada componente se pruebe de forma aislada, se documente con su contrato de props y se reutilice en todas las pantallas (login, dashboard, mascotas, citas, pagos) sin reescribir estilos, garantizando consistencia visual y facilitando el mantenimiento del producto.

Referencias

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO.
- Norman, D. A. (2013). *The design of everyday things* (Rev. ed.). Basic Books.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Anexo A — Código de los componentes (React Native / Expo SDK 56)

Implementación completa de los componentes de UI móviles descritos en el punto 5, en React Native con TypeScript y los tokens del design system de Pawcare (archivo `docs/microtaller-03-componentes.tsx`).

```
/**
 * Pawcare – Diseño de componentes (Microtaller 03)
 * Componentes de UI móviles en React Native (Expo SDK 56, TypeScript).
 * Cada componente parte de su DEFINICIÓN (contrato de props + estados) y se
 * implementa con los tokens compartidos del design system de Pawcare.
 */
import React from 'react';
import {
  View,
  Text,
  Pressable,
  TextInput,
  StyleSheet,
  type PressableProps,
} from 'react-native';

/* =====
Tokens (extracto de src/theme/tokens.ts)
===== */
export const tokens = {
  colors: {
    background: '#FAF8F3',
    foreground: '#1F2A37',
    card: '#FFFFFF',
    primary: '#36B6AC',
    primaryForeground: '#FFFFFF',
    secondary: '#EDF5F4',
    secondaryForeground: '#1E5A53',
    mutedForeground: '#6B7280',
    destructive: '#EF4444',
    success: '#16A34A',
    warning: '#F59E0B',
    border: '#D6E7E4',
  },
  radius: 8,
  radiusFull: 9999,
  spacing: 8,
  touch: 44, // área táctil mínima (Material Design)
} as const;

/* =====
1. Button – acción del usuario
Definición: variante (primary|secondary|outline|destructive|ghost),
tamaño (sm|md|lg), fullWidth opcional, área táctil ≥ 44 px.
```

```

===== */
type ButtonVariant = 'primary' | 'secondary' | 'outline' | 'destructive' |
'ghost';
type ButtonSize = 'sm' | 'md' | 'lg';

interface ButtonProps extends PressableProps {
  label: string;
  variant?: ButtonVariant;
  size?: ButtonSize;
  fullWidth?: boolean;
}

const SIZE_HEIGHT: Record<ButtonSize, number> = { sm: 36, md: 40, lg: 44 };

export function Button({
  label,
  variant = 'primary',
  size = 'lg',
  fullWidth,
  ...rest
}: ButtonProps) {
  const v = buttonVariantStyles[variant];
  return (
    <Pressable
      accessibilityRole="button"
      style={({ pressed }) => [
        styles.btn,
        { minHeight: SIZE_HEIGHT[size], backgroundColor: v.bg, borderColor:
v.border },
        fullWidth && styles.btnBlock,
        pressed && { opacity: 0.9 },
      ]}
      {...rest}
    >
      <Text style={[styles.btnLabel, { color: v.fg }]}>{label}</Text>
    </Pressable>
  );
}

const buttonVariantStyles: Record<ButtonVariant, { bg: string; fg: string; border:
string }> = {
  primary: { bg: tokens.colors.primary, fg: tokens.colors.primaryForeground,
border: 'transparent' },
  secondary: { bg: tokens.colors.secondary, fg: tokens.colors.secondaryForeground,
border: 'transparent' },
  outline: { bg: 'transparent', fg: tokens.colors.foreground, border:
tokens.colors.border },
  destructive: { bg: tokens.colors.destructive, fg: '#FFFFFF', border:
'transparent' },
  ghost: { bg: 'transparent', fg: tokens.colors.primary, border: 'transparent' },
};

/* =====
2. TextField – entrada de datos con validación
Definición: label, placeholder, value, secureTextEntry, error?.

```

```

    fontSize 16 para evitar el zoom automático en iOS.
    ===== */
interface TextFieldProps {
  label: string;
  value: string;
  onChangeText: (text: string) => void;
  placeholder?: string;
  secureTextEntry?: boolean;
  error?: string;
}

export function TextField({ label, value, onChangeText, placeholder,
secureTextEntry, error }: TextFieldProps) {
  return (
    <View style={styles.formGroup}>
      <Text style={styles.label}>{label}</Text>
      <TextInput
        style={[styles.input, !!error && styles.inputError]}
        value={value}
        onChangeText={onChangeText}
        placeholder={placeholder}
        placeholderTextColor={tokens.colors.mutedForeground}
        secureTextEntry={secureTextEntry}
        autoCapitalize="none"
      />
      {!!error && <Text style={styles.fieldError}>{error}</Text>}
    </View>
  );
}

/* =====
3. Badge – etiqueta de estado del dominio
Definición: tone (success|warning|destructive|primary), label.
===== */
type BadgeTone = 'success' | 'warning' | 'destructive' | 'primary';

const BADGE_TONES: Record<BadgeTone, { bg: string; fg: string }> = {
  success: { bg: 'rgba(22,163,74,0.15)', fg: tokens.colors.success },
  warning: { bg: 'rgba(245,158,11,0.18)', fg: '#B45309' },
  destructive: { bg: 'rgba(239,68,68,0.15)', fg: tokens.colors.destructive },
  primary: { bg: tokens.colors.secondary, fg: tokens.colors.secondaryForeground },
};

export function Badge({ label, tone = 'primary' }: { label: string; tone?:
BadgeTone }) {
  const t = BADGE_TONES[tone];
  return (
    <View style={[styles.badge, { backgroundColor: t.bg }]}>
      <Text style={[styles.badgeText, { color: t.fg }]}>{label}</Text>
    </View>
  );
}

/* =====
4. StatCard – métrica numérica del dashboard

```

```

    Definición: value (número grande) + label (texto muted).
    ===== */
export function StatCard({ value, label }: { value: number | string; label: string
}) {
  return (
    <View style={styles.statCard}>
      <Text style={styles.statValue}>{value}</Text>
      <Text style={styles.statLabel}>{label}</Text>
    </View>
  );
}

/* =====
5. AppointmentCard – fila compuesta (avatar + texto + Badge)
Definición: petEmoji, title, subtitle, status, onPress.
Reutilizada en citas del dueño y agenda del administrador.
===== */
interface AppointmentCardProps {
  petEmoji: string;
  title: string;
  subtitle: string;
  status: { label: string; tone: BadgeTone };
  onPress?: () => void;
}

export function AppointmentCard({ petEmoji, title, subtitle, status, onPress }:
AppointmentCardProps) {
  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      style={({ pressed }) => [styles.listRow, pressed && { opacity: 0.9 }]}
    >
      <View style={styles.petAvatar}>
        <Text style={styles.petAvatarText}>{petEmoji}</Text>
      </View>
      <View style={styles.listRowBody}>
        <Text style={styles.listRowTitle} numberOfLines={1}>{title}</Text>
        <Text style={styles.listRowSub} numberOfLines={1}>{subtitle}</Text>
      </View>
      <Badge label={status.label} tone={status.tone} />
    </Pressable>
  );
}

/* =====
Estilos
===== */
const styles = StyleSheet.create({
  btn: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'center',
    borderRadius: tokens.radius,
    borderWidth: 1,

```

```

paddingHorizontal: 16,
},
btnBlock: { alignSelf: 'stretch' },
btnLabel: { fontSize: 15, fontWeight: '600' },

formGroup: { gap: 6, marginBottom: 12 },
label: { fontSize: 14, fontWeight: '600', color: tokens.colors.foreground },
input: {
  minHeight: tokens.touch,
  fontSize: 16, // evita el zoom automático en iOS
  paddingHorizontal: 12,
  backgroundColor: tokens.colors.card,
  color: tokens.colors.foreground,
  borderWidth: 1,
  borderColor: tokens.colors.border,
  borderRadius: tokens.radius,
},
inputError: { borderColor: tokens.colors.destructive },
fieldError: { fontSize: 13, color: tokens.colors.destructive },

badge: { borderRadius: tokens.radiusFull, paddingHorizontal: 10,
paddingVertical: 3, alignSelf: 'flex-start' },
badgeText: { fontSize: 12, fontWeight: '600' },

statCard: { flex: 1, alignItems: 'center' },
statValue: { fontSize: 28, fontWeight: '700', color: tokens.colors.primary },
statLabel: { fontSize: 13, color: tokens.colors.mutedForeground },

listRow: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 12,
  padding: 12,
  backgroundColor: tokens.colors.card,
  borderWidth: 1,
  borderColor: tokens.colors.border,
  borderRadius: tokens.radius,
},
petAvatar: {
  width: 44,
  height: 44,
  borderRadius: tokens.radiusFull,
  backgroundColor: tokens.colors.secondary,
  alignItems: 'center',
  justifyContent: 'center',
},
petAvatarText: { fontSize: 22 },
listRowBody: { flex: 1, minWidth: 0 },
listRowTitle: { fontSize: 16, fontWeight: '600', color: tokens.colors.foreground },
listRowSub: { fontSize: 13, color: tokens.colors.mutedForeground },
});

```